

Functions Cheatsheet

Lessons: [06 — Functions](#)

Examples: [06](#)

Covers: declarations, multiple returns, variadic, closures, function values, `defer` / `panic` / `recover`

Legend: `[*]` = real Go feature that the lesson has not covered yet

DECLARING

<code>func f() {}</code>	no params, no return
<code>func f(a int, b string) {}</code>	typed params
<code>func f(a, b int) {}</code>	shared type, written once
<code>func f() int { return 0 }</code>	one result
<code>func f() (int, error)</code>	several results – the Go signature
<code>func f() (n int, err error)</code>	NAMED results: declared and zero-valued
<code>func f(...) (err error) {}</code>	named result + bare <code>`return`</code> returns its value
<code>func (r T) m() {}</code>	a method – see the interfaces sheet
<code>func f[T any](v T) T</code>	<code>[*]</code> generic function – see the generics sheet
(functions are package-level or literals; no nested func declarations)	

RETURNING MULTIPLE VALUES

<code>return a, b</code>	results in declaration order
<code>v, err := f()</code>	the caller must handle both
<code>_ , err := f()</code>	discard a result with the blank identifier
<code>v, err := f(); if err != nil</code>	the canonical shape
<code>return</code>	bare return with named results ("naked return")
(error is ALWAYS the last result; results are values, not exceptions)	

VARIADIC

<code>func sum(nums ...int) int</code>	<code>nums</code> is an <code>[]int</code> inside the function
<code>sum(1, 2, 3)</code>	pass any number of args
<code>sum()</code>	legal: <code>nums</code> is <code>nil</code> , <code>len 0</code>
<code>sum(slice...)</code>	spread an existing slice
<code>func f(a string, rest ...int)</code>	variadic must be the LAST parameter
<code>fmt.Printf(format, args...)</code>	the <code>stdlib</code> 's most-used variadic shape
(the callee can modify the backing array when you spread a slice)	

CLOSURES

<code>func() { fmt.Println(x) }</code>	a literal capturing <code>x</code> from the enclosing scope
<code>counter := func() func() int</code>	a function returning a function
<code>{ n := 0; return func() int { n++; return n } }</code>	
	each call to the outer <code>func</code> gets its own <code>n</code>
<code>go func(){ ... }()</code>	the goroutine form – captures by reference
<code>defer func(){ ... }()</code>	the defer form – sees final values
callbacks, middleware, options – all built on closures	
(captured variables are shared, not copied; per-iteration since Go 1.22)	

FUNCTIONS AS VALUES

<code>var f func(int) int</code>	a function-typed variable; zero value is nil
<code>f = double</code>	assign a named function
<code>f(3)</code>	call through the variable
<code>type Handler func(w, r)</code>	a named function type
<code>func (f Handler) ServeHTTP()</code>	adapter trick: give a func type a method
<code>func apply(nums []int, fn func(int) int)</code>	take a function as a parameter
<code>sort.Slice(s, func(i, j int) bool { ... })</code>	the classic callback
(calling a nil function value panics)	

defer

<code>defer f()</code>	runs when the FUNCTION returns, however it returns
<code>defer mu.Unlock()</code>	immediately after Lock – the pairing idiom
<code>defer file.Close()</code>	but check the error on writers
<code>defer resp.Body.Close()</code>	after checking err, not before
LIFO order	last registered, first run
args evaluated at defer time	the CALL happens later
<code>defer func(){ ... }()</code>	closure form: reads final variable state
named result + defer	a defer can CHANGE the returned value
(defers still run during a panic – that's the whole point)	

panic & recover

<code>panic("message")</code>	unwind the stack, running defers as it goes
<code>panic(err)</code>	any value works; an error is most useful
<code>recover()</code>	inside a deferred func: stop the panic, get the value
<code>if r := recover(); r != nil</code>	the guard shape
runtime errors panic	nil deref, index out of range, /0 on ints
<code>os.Exit(1)</code>	NOT a panic: no defers, no recover
(recover only works when called DIRECTLY by a deferred function)	

PATTERNS (shape, not API)

Guard clause	<code>if err != nil { return ..., err }</code> – keep the happy path flat
Error wrapping	<code>return fmt.Errorf("load user: %w", err)</code>
Constructor	<code>func New(...) (*T, error)</code> – return a ready-to-use value
Functional option	<code>func WithTimeout(d) Option</code> – see the patterns sheet
Middleware	<code>func(next Handler) Handler</code> – a closure over next
Callback	pass behaviour in as a func parameter
Cleanup	defer at the top, right after acquiring the resource
Recover boundary	recover at a goroutine/handler edge, then log and continue

TRAPS & MEMORIZE

defer in a loop	accumulates to the end of the FUNCTION
defer f(x)	x is captured now; use a closure for the latest value
naked returns	fine in 3 lines, unreadable in 30
recover outside a defer	returns nil and does nothing
recover in a nested func	doesn't stop the panic – must be the deferred one
a panic in a goroutine	crashes the whole program; recover per goroutine
returning a pointer to local	perfectly safe – Go moves it to the heap
nil func value	calling it panics
ignoring the error result	the one habit that hides real bugs
variadic slice spread	sum(s...) can be mutated by the callee